ENSF 380

TOPIC 23: TESTING PART III

Creating Unit Tests

Tests are part of a new class

Same package as code to be tested

Include import statements
- Common are `import org.junit.*;` and `import static org.junit.Assert.*;`

Each test appears as a void method

Use `@Test` annotation before each test

Tests are independent and may be run in any order

Best practice to have one assert statement per test

Naming conventions
- Names of test files usually **end** with “Test”
- Names of individual tests usually **start** with “test”

Example: Testing Pet.java

01_TestingCode

Class Pet Data Members

```
public class Pet {

    private String name;
    private String species;
    private String breed;
    private int numLegs;
    private String colour;
    private String coveringType;
    private final int YEAR;
    private final int MONTH;
    private final int DAY;
```

Class Pet Constructor

```
public Pet (String name, String species,
String breed, int numLegs, String colour,
String coveringType, int[] birthday) {
        this.name = name;
        this.species = species;
        this.breed = breed;
        this.numLegs = numLegs;
        this.colour = colour;
        this.coveringType = coveringType;
        this.YEAR = birthday[0];
        this.MONTH = birthday[1];
        this.DAY = birthday[2];
    }
```

Example: Testing Pet.java

All data members are private

All data members have getters

All non-final data members have setters

All class methods are public

Let’s test public int calculateAge()…

1. What does the method do? What behavior do we expect?
2. Create an instance of the class (new Pet)
3. Call the calculateAge() method and store the result
4. Compare the actual result to the expected result

PetTest.java

01_TestingCode

```
import org.junit.*;
import static org.junit.Assert.*;
import java.time.LocalDate;


public class PetTest {
    int currentYear = LocalDate.now().getYear();


    public PetTest() {
    }
```

PetTest.java

01_TestingCode

```
...
@Test
public void testCalculateAgeBirthdayPast() {
    int[] birthDate = new int[]{2020,1,1};
    Pet petDog = new Pet("Pongo", "Dog", "Dalmatian", 4, "Spotted", "Fur", birthDate);
    int expResult = currentYear - birthDate[0];
    int result = petDog.calculateAge();
    assertEquals("Calculated age was incorrect: ", expResult, result);
}
...
```

AAA

Java tests follow the pattern: Arrange, Act, Assert

Arrange: take steps to be able to generate the value you will test

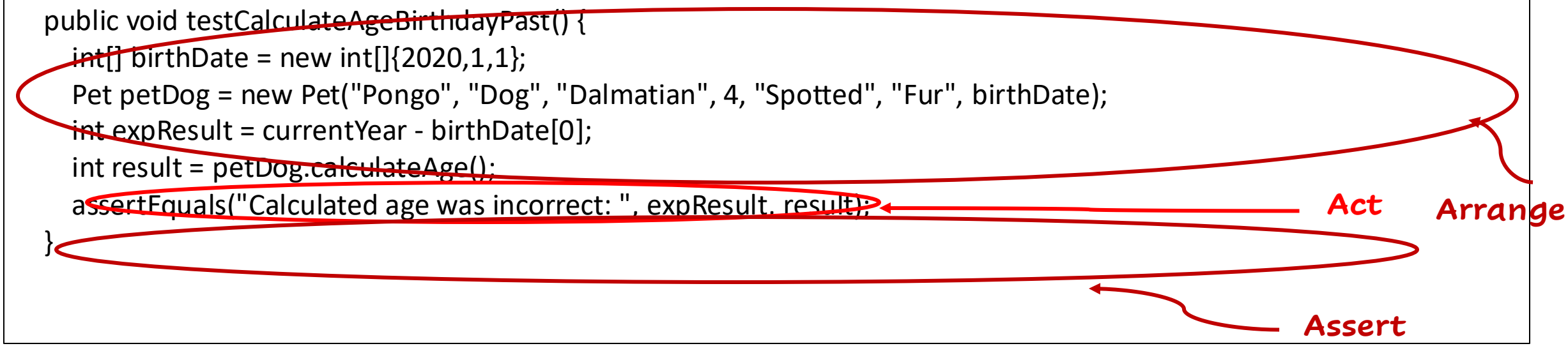
Act: generate the value you will test

Assert: make a statement comparing the value with the expected value

PetTest.java

01_TestingCode

```
...
@Test
public void testCalculateAgeBirthdayPast() {
    int[] birthDate = new int[]{2020,1,1};
    Pet petDog = new Pet("Pongo", "Dog", "Dalmatian", 4, "Spotted", "Fur", birthDate);
    int expResult = currentYear - birthDate[0];
    int result = petDog.calculateAge();
    assertEquals("Calculated age was incorrect: ", expResult, result);
}
...
```



First Test: Birthday has already occurred this year

Method calculateAge()

01_TestingCode/Pet.java

```
public int calculateAge(){

  ZoneId timeZone = TimeZone.getTimeZone("GMT-
7").toZoneId();

  LocalDate currentDate = LocalDate.now(timeZone);

  int currentYear = currentDate.getYear();

  int currentMonth = currentDate.getMonthValue();

  int currentDay = currentDate.getDayOfMonth();

  int age = currentYear - YEAR;

  return age;

}
```

Test #1 for calculateAge()

01_TestingCode/PetTest.java

```
    @Test

    public void testCalculateAgeBirthdayPast() {

        int[] birthDate = new int[]{2020,1,1};

        Pet petDog = new Pet("Pongo", "Dog", "Dalmatian", 4, "Spotted", "Fur",
birthDate);

        int expResult = currentYear - birthDate[0];

        int result = petDog.calculateAge();

        assertEquals("Calculated age was incorrect: ", expResult, result);

    }
```

Second Test: Birthday has not already occurred this year

Method calculateAge()

01_TestingCode/Pet.java

```
public int calculateAge(){

  ZoneId timeZone = TimeZone.getTimeZone("GMT-
7").toZoneId();

  LocalDate currentDate = LocalDate.now(timeZone);

  int currentYear = currentDate.getYear();

  int currentMonth = currentDate.getMonthValue();

  int currentDay = currentDate.getDayOfMonth();

  int age = currentYear - YEAR;

  return age;

}
```

Test #2 for calculateAge()

01_TestingCode/PetTest.java

```
  @Test

  public void testCalculateAgeBirthdayFuture() {

    int[] birthDate = new int[]{2020,12,30};

    Pet petDog = new Pet("Pongo", "Dog", "Dalmatian", 4, "Spotted", "Fur",
birthDate);

    int expResult = currentYear - birthDate[0] - 1;

    int result = petDog.calculateAge();

    assertEquals("Calculated age was incorrect: ", expResult, result);

  }
```

Third Test: Birthday occurs today

Method calculateAge()

01_TestingCode/Pet.java

```
public int calculateAge(){

  ZoneId timeZone = TimeZone.getTimeZone("GMT-
7").toZoneId();

  LocalDate currentDate = LocalDate.now(timeZone);

  int currentYear = currentDate.getYear();

  int currentMonth = currentDate.getMonthValue();

  int currentDay = currentDate.getDayOfMonth();

  int age = currentYear - YEAR;

  return age;

}
```

Test #3 for calculateAge()

01_TestingCode/PetTest.java

```
  @Test

  public void testCalculateAgeBirthdayToday() {

    int[] birthDate = new int[]{2020,12,25};

    Pet petDog = new Pet("Pongo", "Dog", "Dalmatian", 4, "Spotted", "Fur",
birthDate);

    int expResult = currentYear - birthDate[0];

    int result = petDog.calculateAge();

    assertEquals("Calculated age was incorrect: ", expResult, result);

  }
```

Demo

RUN TESTS

Test Results: Two successes, one failure

01_TestingCode/Pet.java

02_FirstRevision/PetTest

SUCCESS: testCalculateAgeBirthdayPast, testCalculateAgeBirthdayToday

FAILURE: testCalculateAgeBirthdayFuture – “Calculated age was incorrect: expected:<3> but was:<4>”

```
There was 1 failure:
1) testCalculateAgeBirthdayFuture(edu.ucalgary.oop.PetTest)
java.lang.AssertionError: Calculated age was incorrect:  expected:<3> but was:<4>
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.failNotEquals(Assert.java:835)
        at org.junit.Assert.assertEquals(Assert.java:647)
        at edu.ucalgary.oop.PetTest.testCalculateAgeBirthdayFuture(PetTest.java:35)

FAILURES!!!
Tests run: 3,  Failures: 1
```

Revise the Code

02_FirstRevision

```
public int calculateAge(){

        ZoneId timeZone = TimeZone.getTimeZone("GMT-7").toZoneId();
        LocalDate currentDate = LocalDate.now(timeZone);

        int currentYear = currentDate.getYear();
        int currentMonth = currentDate.getMonthValue();
        int currentDay = currentDate.getDayOfMonth();

        int age = currentYear - YEAR;

        if (currentMonth < MONTH && currentDay < DAY)
            age--;

        return age;
    }
```

Test Choice Matters

The test would still fail if we ran it any day before the 30th of the month

The test would succeed if we ran it on the 31st of the month

Tests will not necessarily catch every bug, which is why we need many tests

- Especially edge cases
- Especially boundary conditions

```
1) testCalculateAgeBirthdayFuture(edu.ucalgary.oop.PetTest)
java.lang.AssertionError: Calculated age was incorrect:  expected:<3> but was:<4>
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.failNotEquals(Assert.java:835)
        at org.junit.Assert.assertEquals(Assert.java:647)
        at edu.ucalgary.oop.PetTest.testCalculateAgeBirthdayFuture(PetTest.java:35)

FAILURES!!!
Tests run: 3,  Failures: 1
```

Revise the Code Again

03_SecondRevision

```
public int calculateAge(){

        ZoneId timeZone = TimeZone.getTimeZone("GMT-7").toZoneId();
        LocalDate currentDate = LocalDate.now(timeZone);

        int currentYear = currentDate.getYear();
        int currentMonth = currentDate.getMonthValue();
        int currentDay = currentDate.getDayOfMonth();

        int age = currentYear - YEAR;

        if (currentMonth < MONTH || (currentMonth == MONTH && currentDay < DAY))
            age--;

        return age;
    }
```

Exercise 23.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Testing Expected Exceptions

04_Exceptions

```
public Pet (String name, String species, String breed, int numLegs, String colour, String coveringType,
int[] birthday) {
        this.name = name;
        this.species = species;
        this.breed = breed;
        this.numLegs = numLegs;
        this.colour = colour;
        this.coveringType = coveringType;

        ZoneId timeZone = TimeZone.getTimeZone("GMT-7").toZoneId();
         LocalDate currentDate = LocalDate.now(timeZone);
         LocalDate birthDate = LocalDate.of(birthday[0], birthday[1], birthday[2]);
         int daysOld = Period.between(birthDate, currentDate).getDays();

         if(daysOld < 0){
            throw new IllegalArgumentException("Pet cannot be registered until born.");
        }
         else{
            this.YEAR = birthday[0];
            this.MONTH = birthday[1];
            this.DAY = birthday[2];
         }
    }
```

Testing Expected Exceptions

04_Exceptions

```
@Test(expected = IllegalArgumentException.class)
public void testCalculateAge_BirthdayException() {
    // Future date two years in the future
    int[] birthDate = new int[]{today.getYear() + 2, 7, 1};
    Pet petDog = new Pet("Pongo", "Dog", "Dalmatian", 4, "Spotted", "Fur", birthDate);
}
```

```
JUnit version 4.13.2
....
Time: 0.028

OK (4 tests)
```

Common Assert Methods

`void assertEquals(String message, primitive type expected, primitive type actual)`
- Checks that two primitive data types are equal, has many overloaded options

`void assertEquals(Object expected, Object actual)`
- Checks that two objects are equal

`void assertNotNull(Object object)`
- Checks that an object is not null

`void assertSame(Object expected, Object actual)`
- Checks that two objects refer to the same object

`void assertTrue(boolean condition)`
- Passes if condition is true

`void assertFalse(boolean condition)`
- Passes if condition is false

and many more (including array operations)…

String messages are optional, but are very useful! (Mandatory in this class)

Can often check for reverse conditions as well (e.g. null vs. not null)

Tip: Start here to find available test options! https://junit.org/junit4/javadoc/latest/org/junit/Assert.html

Testing Implementation Details

Create objects using constructors, then test if null

Test deep vs. shallow copies using “same objects”

`java.lang.reflect` package can be useful:

- Get the name of a class
- Get the name of a method
- Get the arguments of a method
- Get the return type of a method
- Check if an interface is implemented

Setup and Teardown

06_MoreTests

@Before – method that runs before each test case
- Example: constructing a complicated object using the same data across multiple tests

@After – method that runs after each test case, regardless of success or failure
- Example: removing a file which may have been written out by a test

@BeforeClass – method that runs only once before any @Before or @Test code is executed
- Example: creating a file that is read by multiple tests

@AfterClass – method that runs only once after all other annotations have been executed
- Example: removing the file that was used by multiple tests

Tip: More information available under JUnit 4 FAQ list: https://junit.org/junit4/faq.html

Exercise 23.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Testing Checklist for Well-Written Tests

Each test should:
Be executable (includes @Test and can be run on the command-line)
Have at least one assert statement or expected exception (list of common assert methods provided previously)
Test only one method
Test a limited number of behaviors (if there are many asserts, split into different test cases unless there are dependencies)
Be descriptively named and commented (including failure messages)
If there is redundant setup/takedown, extract into common methods (e.g., @Before, @After)

Testing Checklist for Test Suites

The test suite should:
Have at least one test per requirement
Appropriately use setup and teardown code (e.g., @BeforeClass, @AfterClass)
Contain a fault-revealing test for each bug in the code (if there is a bug, there should be a failing test)
For each requirement, include tests for: • valid inputs • boundary conditions and edge cases • invalid inputs • expected exceptions
Be grouped by class, and by class relationships

Exercise 23.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Test creation using Arrange, Act, Assert

Testing for expected exceptions

Test assertions

Test setup and teardown methods

Considerations for writing a well-written unit test

Considerations for creating a good test suite

org.junit is the first library outside of the java namespace which we've introduced in this course. You are permitted to make use of this package within your code.

However, remember that you should never import classes or packages in files when they are not used. Including an import statement for (for example) org.junit.* in a non-test file is not appropriate.

Summary: Skills

Creating simple tests

Testing with assertions and expected exceptions

Testing and debugging existing code

Additional Information

Related reading

- Volume 2, Chapter 6.2: Local Dates
- Volume 2, Chapter 6.5: Zoned Time